TGSL: Temporal Graph Structure Learning for Stock Market Prediction

Nima Tavassoli
University of Tehran
Tehran, Iran
nima.tavassoli@ut.ac.ir

Saman Haratizadeh
University of Tehran
Tehran, Iran
haratizadeh@ut.ac.ir

Davide Mottin
Aarhus University
Aarhus, Denmark
davide.mottin@cs.au.dk

Abstract

Graph Neural Networks (GNNs) are a natural fit for stock market prediction, where a stock's behavior depends on its interactions with others, but they only reach their potential when the underlying graph reflects the true relations between stocks. No such ground-truth graph exists: prior graph-based predictors hand-craft a *fixed* structure from naive statistics (e.g., correlation) that is decoupled from the task and cannot evolve over time. We cast market-graph construction as a graph structure learning problem and propose TGSL, a model that jointly learns, end-to-end, a *daily* graph of a set of stocks and a GNN classifier of each stock's next-day movement. TGSL combines the temporal information of each stock (via recurrent encoders) with its structural information (via a graph encoder), and disentangles structure from feature generation in two parallel branches. On 93 large-cap NASDAQ stocks, TGSL improves accuracy by 1% over the strongest baseline, is robust to the initial graph it is seeded with, and learns sparse, time-varying, label-homophilic structures, showing that learning the graph, rather than fixing it, helps both accuracy and interpretability.

1 Introduction

The stock market, as a dynamic and complex financial ecosystem, has long captivated the interest of investors, economists, and researchers [26]. Understanding and predicting price movements in this domain is of paramount importance, as it directly impacts investment decisions, risk management, and financial stability. The movement of stock prices is influenced by a multitude of factors, including economic indicators, geopolitical events, news sentiment, and, most notably, the intricate relationships that exist between different stocks. Because the volume of relevant data is enormous and many factors act on prices either directly or indirectly, producing accurate forecasts is extremely challenging, and a wide range of models, from machine learning to game theory and dynamical systems, have been brought to bear on the problem.

In recent years, network modeling of stocks has emerged as a promising approach to capture these kinds of relations, due to the fact that understanding the behavior of an economy requires examining the interactions among its various components rather than studying them in isolation [4]. By constructing graphs that represent the connections between stocks, we gain the ability to extract hidden patterns and dependencies, and we can further leverage external knowledge bases to augment the structural insights derived from these graphs. Such network modeling has been used both to *analyze* markets, e.g., to find influential stocks or compute centrality measures, and to *predict* the future behavior of stocks treated as nodes in a graph [10, 4].

Graph Neural Networks (GNNs) have become an important tool for processing graph-based datasets by considering both the underlying graph structure and the attributes associated with nodes (or edges) [37]. GNNs generate node embeddings by aggregating information from neighboring nodes, and as a result, having access to an accurate structure significantly influences the quality of node representations. However, in real-world scenarios there are cases in which the provided graph is incomplete or noisy; indeed, even imperceptible perturbations, so-called *adversarial attacks*, can easily fool a GNN [36]. The stock market graph, which aims to model stocks as nodes in a graph, falls squarely into this category, as there is no clear and definitive ground-truth structure that comprehensively captures the relationships among stocks. Prior research has often relied on the manual construction of the input structure through correlation or causality calculations, frequently involving a trial-and-error process [10, 13, 12]. These structures are typically computed from low-level stock features such as price; they may not be optimally suited for the downstream prediction task, and, being static, they lack the capability to adapt and evolve as time progresses. We refer to this mismatch as the gap between the *raw* graph and the *optimal* graph for prediction.

Hence, we propose TGSL, a model that learns an optimal graph structure for a set of stocks by jointly exploiting their temporal and structural information, depicted in Figure 1. TGSL follows the deep graph structure learning paradigm [38], in which the adjacency matrix is parameterized with learnable weights and trained, end-to-end and under the supervision of a downstream task, together with the parameters of a GNN. Concretely, we begin from a (possi-

bly empty) initial graph for a given set of stocks, encode a sliding window of past daily graphs with a shared graph encoder, and feed the resulting temporal sequences into two parallel recurrent branches: one that builds the day-specific adjacency matrix and one that builds the node features for classification. A GNN classifier then predicts the next-day movement of each stock from the learned structure and features. To the best of our knowledge, this is the first work to apply graph structure learning to financial market prediction, where the constructed graph is allowed to differ from day to day rather than remaining fixed over time.

Our experiments, conducted on a dataset of 93 NASDAQ stocks with the highest market capitalization, reveal that TGSL enhances next-day movement prediction accuracy by 1% compared to the state-of-the-art baseline. Moreover, we show that the accuracy of TGSL is essentially independent of the initial graph it is seeded with: it can start from isolated nodes and still recover a competitive structure, whereas the predictive variants of our framework remain sensitive to the input graph, underscoring the value of the structure-learning component. An analysis of the learned graphs further shows that they are sparse, time-varying, and homophilic with respect to price-movement labels.

Contributions. In summary, this paper makes the following contributions:

- We formulate market-graph construction for stock movement prediction as a *graph structure learning* problem, and present TGSL, an end-to-end model that jointly learns a day-specific graph and a GNN predictor from both the temporal and structural information of stocks.
- We disentangle structure generation from feature generation into two parallel recurrent branches, and we introduce a label-aware target-smoothing regularizer that sparsifies the learned graph while improving classification.
- We study three progressively simpler variants of the framework that ablate its temporal and structural components, clarifying the contribution of each design choice.
- We show on real NASDAQ data that TGSL outperforms strong graph-based and graph-structure-learning baselines, is robust to the choice of the initial graph, and produces interpretable, time-varying market structures.

2 Related Work

Existing work related to ours can be grouped into three categories: classical and deep predictors for financial markets, graph-based stock market prediction, and graph structure learning.

Stock market prediction. The literature on forecasting financial markets is vast [26]. Classical approaches rely on traditional machine-learning models such as random forests, support vector machines, naive Bayes, and k-nearest neighbors applied to historical prices and technical indicators [15, 22, 27]. A recurring finding is that the choice and number of technical indicators strongly affects accuracy, and that combining heterogeneous data sources (e.g., news sentiment) with price data tends to help [9, 24]. With the advent of deep learning, recurrent architectures, in particular Long Short-Term Memory (LSTM) networks, became dominant because of the sequential nature of stock data, often augmented with attention mechanisms [25]. CNNpred [8] instead applies convolutional neural networks to a diverse set of financial variables, organized as 2D and 3D tensors, and aggregates information across several related markets to predict a target market. These methods, however, treat each asset (or market) largely in isolation and do not explicitly model the relations between assets.

Graph-based stock market prediction. Network modeling for stock prediction is a relatively recent approach [4], where typically each stock is treated as a node and the edges encode relationships between pairs of stocks. The *correlation network* is the most commonly employed construction: edges are weighted by a correlation coefficient (Pearson, Spearman, or Kendall) computed between the time series of stock pairs [17, 20, 19]. Because the resulting graph is complete, many works filter it with a maximum spanning tree to keep only the most important edges [12]. Beyond correlation, several works build *causality* or information-theoretic networks using transfer entropy [23, 21] or (partial) mutual information [6, 35] to capture nonlinear, directed influence between assets. Patil *et al.* [19] additionally derive a co-mention graph from financial news, connecting stocks that appear together in the same article. Peng [20] compares three stock graphs (correlation, sector, and dynamic time warping) and reports that the correlation graph performs best, while noting that clipping correlations to non-negative weights discards potentially useful negative correlations.

A second line of work focuses on the predictor built on top of a fixed graph. HATS [13] constructs a relational graph between companies from a knowledge base (e.g., Wikidata) and uses a hierarchical graph attention network that learns the importance of neighbors and of relation types. Price Graphs [30] converts the price time series of each stock into a *visibility graph*, extracts structural embeddings, and predicts the trend with recurrent and attention layers. Other works build correlation- or sector-based graphs and run GCN/GAT predictors over them [34, 32]. Because labeled samples are scarce, graph-based *semi-supervised* learning is common: Kia *et al.* [12] propagate labels over a correlation network of market indices, exploiting time-zone lags between markets, and report that a better network structure improves accuracy. The closest baseline to our work, GCNET [10], introduces an *influence network* (built by checking, for each node, whether adding a neighbor's features improves a linear classifier) and trains a GCN semi-supervisedly with *plausible* (pseudo) labels generated by classical models for unlabeled nodes. We reuse GCNET's

influence graph and its plausible-label discovery procedure as components in our pipeline. Crucially, all of these methods rely on a *predefined, static* graph that is decoupled from the prediction objective; in contrast, we *learn* a day-specific graph driven directly by the prediction loss.

Graph structure learning. Graph structure learning (GSL) aims to jointly learn an optimal graph structure and the associated GNN parameters, typically under the supervision of a downstream task [38]. Methods are commonly organized by how they model the structure [38]: (i) *metric learning*, where an edge weight is a learned distance/similarity between the two endpoints' embeddings, optionally followed by post-processing (e.g., k-NN sparsification or symmetrization). Kernel- and attention-based similarities are typical [28, 2]; for instance, Cosmo *et al.* [2] use a multi-layer perceptron to produce edge weights, and Egilmez *et al.* [3] use cosine distance. A limitation of metric-learning methods is that they refine the weights of existing relations but cannot, on their own, create entirely new edges. (ii) *Probabilistic modeling*, where edges are assumed to be sampled from a distribution whose parameters are learned; the main challenge is to make sampling differentiable. LDS [7] models edges as Bernoulli variables and optimizes their parameters jointly with the GCN through a bilevel formulation. (iii) *Direct optimization*, where the adjacency matrix itself is treated as a free parameter learned alongside the GNN, often guided by graph regularizers that explicitly encode desired properties such as sparsity, smoothness, or low rank. IDGL [1] alternates between structure modeling and message passing using spectral and sparsity priors; Yang *et al.* [33] use node labels to connect same-class nodes; Pro-GNN [11] and low-rank methods [31] regularize the learned adjacency to be robust to noise and attacks. Two methods are particularly relevant. SLAPS [5] observes that edges far from any labeled node receive no supervision ("supervision starvation") and adds a self-supervised denoising auto-encoder task to guide structure learning. SUBLIME [16] learns the structure in a fully unsupervised manner via contrastive learning between an anchor view and a learner view, together with a structure-bootstrapping mechanism.

Notably, GSL has been studied almost exclusively on *static* benchmarks such as citation networks, and prior evaluations seldom consider temporal data. A handful of works consider time-varying structures in other domains, e.g., dynamic structure learning for soil moisture forecasting [29]. Our work bridges these two literatures: we bring deep graph structure learning to financial market prediction, learning a *temporal*, day-specific structure tailored to the prediction task. Tables comparing the surveyed structure-learning and graph-based prediction methods are summarized in the discussion of Section 5.

3 Background and Problem

We first review the building blocks used by our model and then formalize the stock graph structure learning problem.

3.1 Preliminaries

Graph neural networks. A graph is denoted by $G = (V, E)$ with a set of nodes V ($n = |V|$) and edges E, summarized by an adjacency matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ and a node feature matrix $\mathbf{X} \in \mathbb{R}^{n \times f}$. A graph neural network (GNN) computes a representation of each node by iteratively aggregating information from its neighbors [37]. For the Graph Convolutional Network (GCN) [14], one propagation layer is

$$\mathbf{H}^{(l+1)} = \sigma\Big(\hat{\mathbf{D}}^{-\frac{1}{2}}\,\hat{\mathbf{A}}\,\hat{\mathbf{D}}^{-\frac{1}{2}}\,\mathbf{H}^{(l)}\,\mathbf{W}^{(l)}\Big), \qquad (1)$$

where $\hat{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ is the adjacency with self-loops, $\hat{\mathbf{D}}$ is its degree matrix, $\mathbf{H}^{(l)}$ is the node representation at layer l (with $\mathbf{H}^{(0)} = \mathbf{X}$), $\mathbf{W}^{(l)}$ is a learnable weight matrix, and $\sigma(\cdot)$ is a non-linear activation. Stacking L layers lets each node aggregate information from its L-hop neighborhood. The Graph Attention Network (GAT) [28] replaces the fixed normalization in Eq. (1) with learnable attention coefficients α_{ij} computed for each edge from the endpoints' transformed features, so that the contribution of a neighbor depends on its learned importance. Both operators only refine representations *given* a structure; their output quality is therefore tightly coupled to the quality of $\mathbf{A}$.

Recurrent encoders. Stock data is inherently sequential, so we encode temporal information with recurrent neural networks (RNNs), which feed the output of one step as input to the next. We use the Long Short-Term Memory (LSTM) network [18], which maintains a cell state regulated by input, forget, and output gates and is able to capture long-range temporal dependencies. Given a sequence of per-day vectors for a node, we take the last hidden state of the LSTM as the node's temporal summary. The Gated Recurrent Unit (GRU) is a lighter variant that merges the forget and input gates; either can be used interchangeably in our model.

3.2 Problem formulation

We are given a set of n stocks $\mathcal{S} = \{s_1, s_2, \ldots, s_n\}$ observed over a sequence of trading days. For each stock s_i and each day t we extract a feature vector $\mathbf{x}_i^{(t)} \in \mathbb{R}^f$ from its market data (e.g., open/close/high/low prices, volume) and technical indicators (Section 5). The next-day price movement of stock s_i at day t defines a binary label $y_i^{(t)} \in \{0, 1\}$, where 1 (resp. 0) denotes that the next-day closing price is higher (resp. not higher) than the current one.

A *stocks graph* for day t is $G^{(t)} = (V, E^{(t)})$ where the node set V represents the stocks and the edges $E^{(t)}$ encode

(signed or weighted) relations between pairs of stocks, summarized by an adjacency matrix $\mathbf{A}^{(t)} \in \mathbb{R}^{n \times n}$. Unlike prior work, we do not assume that a single, time-invariant $\mathbf{A}$ is given; instead we wish to construct, for every day, a graph that is most useful for predicting that day's labels.

Problem 1 (Stocks graph structure learning) *Given the per-day feature matrices of a window of the T most recent days, $\{\mathbf{X}^{(t-T+1)}, \ldots, \mathbf{X}^{(t)}\}$ with $\mathbf{X}^{(t)} \in \mathbb{R}^{n \times f}$, and optionally an initial structure $\mathbf{A}_0$, learn a structure-generation function that outputs a day-specific adjacency matrix $\mathbf{A}^{(t)} \in \mathbb{R}^{n \times n}$, a feature matrix $\mathbf{X}^{(t)} \in \mathbb{R}^{n \times d}$, and a GNN classifier $f : \mathbb{R}^{n \times n} \times \mathbb{R}^{n \times d} \to [0,1]^n$, such that the predicted labels $\hat{\mathbf{y}}^{(t)} = f(\mathbf{A}^{(t)}, \mathbf{X}^{(t)})$ match the true next-day movement labels $\mathbf{y}^{(t)}$.*

The structure-generation function and the classifier f are trained jointly, end-to-end, under the supervision of the prediction task. This is in contrast to graph-based predictors that fix $\mathbf{A}$ a priori (e.g., from correlation or causality) and only train f. Because $\mathbf{A}^{(t)}$ is produced from the temporal features of day t, the learned graph naturally varies over time. In the next section we describe TGSL, which instantiates this formulation, together with three simpler variants that ablate its components.

4 The TGSL Model

We design TGSL to simultaneously learn a day-specific adjacency matrix and the GNN parameters for predicting next-day stock price movement. As presented in Figure 1, the TGSL model consists of an initializer GNN that encodes a window of past daily graphs, followed by two parallel recurrent branches: one that generates the learned adjacency matrix $\mathbf{A}$ and one that generates the node features $\mathbf{X}$. A classifier GNN then predicts the next-day movement label of every node (stock) from $(\mathbf{A}, \mathbf{X})$. We first describe the components of TGSL (Section 4.1), then its loss function and training (Section 4.2), and finally three simpler variants used in our ablation (Section 4.3).

4.1 TGSL Components

Initializer GNN. As input, we take a window of initial graphs $\mathcal{G} = \{G^{(t-T+1)}, \ldots, G^{(t)}\}$ of size T, where $G^{(\tau)}$ is the graph of stocks for day τ. All graphs in the window share a common structure, a fixed initial graph $\mathbf{A}_0$ that can be chosen among the constructions used in prior work (e.g., correlation or influence network), or even an empty graph, but they differ in their node features. The initializer GNN is a two-layer network whose weights are *shared* across all T graphs in the window; applied to each day, it produces a structure-aware embedding of every node. At the end of this stage, each stock has T feature vectors $\mathbf{v}^{(\tau)} \in \mathbb{R}^d$, one per day τ, that summarize both its own attributes and its position in the (initial) market structure.

Structure-learning branch. After deriving the per-day node embeddings, we treat the T vectors of each node as a temporal sequence. To exploit the sequential characteristics of this signal, we use a two-layer LSTM that receives all node embeddings $\mathbf{V} \in \mathbb{R}^{n \times T \times d}$ from the previous step and outputs, as the updated embedding of each node, its last hidden state $\mathbf{u} \in \mathbb{R}^{d'}$. Stacking these gives an embedding matrix $\mathbf{U} \in \mathbb{R}^{n \times d'}$. Based on the updated embeddings we construct a similarity matrix by the inner product of every node pair,

$$\tilde{\mathbf{A}} = \mathbf{U}\,\mathbf{U}^\top, \tag{2}$$

which models each edge weight as the similarity between its endpoints. This aligns with the network homophily assumption, which posits that similar nodes are more likely to be connected. Because the inner product yields both positive and negative values, and because $\tilde{\mathbf{A}}$ is in general neither normalized nor symmetric, we post-process it into a valid adjacency matrix:

$$\mathbf{A} \;=\; \tfrac{1}{2}\,\mathbf{D}^{-\frac{1}{2}}\Big(P(\tilde{\mathbf{A}}) + P(\tilde{\mathbf{A}})^\top\Big)\,\mathbf{D}^{-\frac{1}{2}}, \tag{3}$$

where P is an element-wise non-linearity with non-negative range (e.g., ReLU) and $\mathbf{D}$ is the degree matrix used to normalize $\mathbf{A}$. The symmetrization term $\frac{1}{2}\big(P(\tilde{\mathbf{A}}) + P(\tilde{\mathbf{A}})^\top\big)$ is only required when the matrix is sparsified with a k-nearest-neighbor graph: if v_j is among the k nearest neighbors of v_i and vice versa, averaging leaves their connection unchanged; if only one direction holds, averaging halves the weight and distributes it symmetrically. The dot-product graph is otherwise complete and is sparsified by the regularizer described below.

Feature-learning branch. In parallel, a second LSTM with the same architecture processes the same sequence of node embeddings to produce the node features $\mathbf{X}$ used for classification. Disentangling the node features that feed the classifier from those that build the adjacency matrix gives the model more flexibility: if a single network produced both, then any two stocks with similar features would necessarily be connected with a large weight, whereas a stock with relatively distant features may still be informative for prediction over a given time window.

Classifier GNN. Finally, a two-layer GCN performs node classification given the learned adjacency matrix $\mathbf{A}$ and feature matrix $\mathbf{X}$. A single layer computes

$$\mathbf{h}_i^{(l+1)} = \sigma\Big(\mathbf{b} + \sum_{j \in \mathcal{N}(i)} e_{ji}\,\mathbf{h}_j^{(l)}\,\mathbf{W}^{(l)}\Big), \tag{4}$$

where e_{ji} is the (numeric) weight of the edge connecting node j to node i, $\mathbf{h}_j^{(l)}$ is the normalized message of node

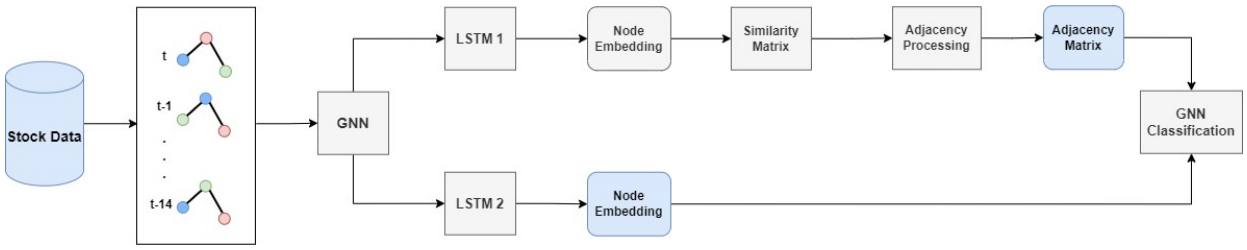


Figure 1: Overview of TGSL. A window of T daily graphs of the same set of stocks is encoded by a shared initializer GNN. Two parallel LSTM branches then process the resulting temporal sequences: the upper branch builds a similarity matrix that, after adjacency processing, yields the day-specific adjacency matrix $\mathbf{A}$; the lower branch produces the node features $\mathbf{X}$. A classifier GNN predicts the next-day movement of each stock from $(\mathbf{A}, \mathbf{X})$.

j at layer l, and $\mathbf{b}$ is a bias. The output layer returns, for each node, the probability of belonging to each of the two movement classes.

4.2 TGSL Loss Function

The main objective of the model is to minimize the prediction error of the classifier, for which we use the cross-entropy loss $\mathcal{L}_{\text{Prediction}}$ over the labeled nodes. Since the dot-product graph of Eq. (2) is complete, we additionally regularize the learned structure. We consider two regularizers.

Target smoothing. Following the intuition that message passing between same-label nodes helps node classification, we penalize edges connecting two nodes with *different* labels:

$$\mathcal{L}_{\text{TS}} = \sqrt{\sum_{i,j\,(i\neq j)} \left(\mathbf{A}_{ij}\,|s_i - s_j|\right)^2}, \tag{5}$$

where s_i is the (binary) label of node i. Since labels are in $\{0, 1\}$, the term $|s_i - s_j|$ acts as a mask that keeps only the weights of edges between differently labeled nodes; minimizing the resulting norm both improves classification and, in practice, drives the adjacency matrix toward sparsity.

Sparsity. Alternatively, a generic sparsity penalty on the adjacency matrix can be used,

$$\mathcal{L}_{\text{sp}}(\mathbf{A}) = \|\mathbf{A}\|_2, \tag{6}$$

whose minimization shrinks the entries of $\mathbf{A}$ and thereby sparsifies the graph; this regularizer has also been used in prior structure-learning work.

Total loss. The overall objective combines the prediction loss with one of the regularizers above,

$$\mathcal{L}_{\text{Total}} = \mathcal{L}_{\text{Prediction}} + \alpha\,\mathcal{L}_{\text{Regularizer}}, \tag{7}$$

where α controls the strength of the regularizer and the regularizer is chosen according to the input data. All operators in the architecture are differentiable, so the whole model is trained end-to-end by gradient descent, updating the parameters of every network simultaneously.

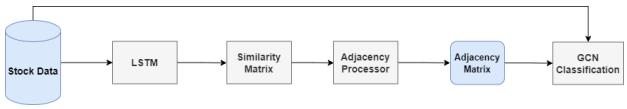
Training and inference. TGSL is trained in a supervised fashion on days whose true labels are available; after training, the model has learned how to turn the last T days of a set of stocks into a graph structure that performs well on node classification. At test time, the best results are obtained by continuing to train *only* the classifier GNN on the test day in a semi-supervised manner, freezing the weights of the other networks, using *plausible labels* for a subset of nodes whose true next-day labels are not yet available. The plausible labels are produced by the procedure of [10], described in Section 5.

4.3 Model Variants

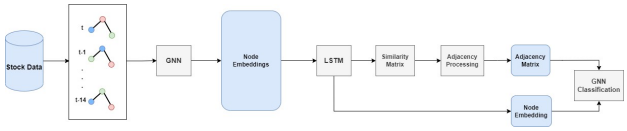
To clarify the contribution of each component, we study three progressively simpler variants of TGSL.

TGSL-S2 (single-LSTM). This variant simplifies TGSL by using a *single* LSTM instead of two (Figure 2b). The window of graphs is first encoded by a shared initializer GNN, and the single LSTM then summarizes the temporal sequence of node embeddings; its output is used along two paths: an upper path that builds the similarity matrix and, after adjacency processing, the adjacency matrix, and a lower path that is fed, unchanged, as node features to the classifier GNN. Comparing TGSL-S2 with TGSL isolates the effect of the second LSTM, i.e., of disentangling structure and feature generation.

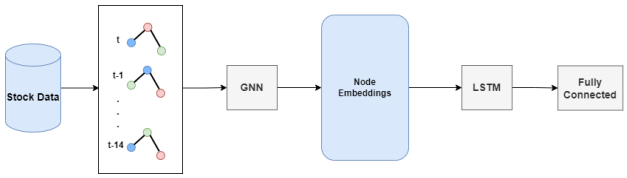
TGSL-S1 (feature-only). The simplest structure-learning variant takes only the stock feature matrix as input, no initial graph and no initializer GNN (Figure 2a). Its core is a *generator* function $G : \mathbb{R}^{n\times f} \to \mathbb{R}^{n\times n}$ with parameters θ_G that maps the node features to an adjacency matrix. The generator can be realized in different ways: full parameterization optimizes the adjacency directly as free parameters ($\theta_G \in \mathbb{R}^{n\times n}$); an MLP, as in SLAPS [5], maps node features to a new space. Because we wish to exploit the temporal features of each stock, we use an LSTM as the generator: it receives the last t days of each stock, produces a node embedding, and the similarity matrix $\tilde{\mathbf{A}} = \mathbf{U}\mathbf{U}^\top$ is post-processed as in Eq. (3). A classifier GNN then performs node classification. TGSL-S1 can be trained semi-supervisedly (using



(a) TGSL-S1: feature-only, single-LSTM generator.



(b) TGSL-S2: graph-window, single LSTM.



(c) TGSL-P: prediction-only, no structure learning.

Figure 2: The three variants of TGSL used in our ablation.

plausible labels) or supervisedly. When trained supervisedly, the optimal graphs produced for past days can be reused for prediction in several ways, e.g., averaging the graphs of the previous few days, taking the union of past adjacency matrices, inferring the graph for the target day from the trained model, or continuing to train the previous day's model semi-supervisedly on the target day; we compare these strategies in Section 5.

TGSL-P (prediction-only). This variant performs *no* structure learning and serves purely as a predictor (Figure 2c). From a fixed, predefined input graph it extracts structural information of the nodes with a GNN, summarizes the resulting sequence with an LSTM, and predicts each stock's next-day movement with a fully connected layer. It thus uses both structural and temporal information but with a static graph, and serves to quantify the benefit of generating an optimal structure (versus fixing it) and the benefit of network modeling over single-stock prediction.

5 Experiments

In this section we evaluate TGSL on next-day stock movement prediction. We first introduce the dataset and preprocessing (Sections 5.1–5.3), then the evaluation metrics, baselines, and implementation (Sections 5.4–5.6). We then report results (Section 5.7), discuss the ablation across our variants (Section 5.8), and analyze the learned graphs (Section 5.9).

5.1 Dataset

To evaluate the proposed models and the baselines, we collected the 100 stocks with the highest market capitalization in the NASDAQ index. Seven stocks without enough history for the training procedure were removed, leaving $n = 93$ stocks. For each stock and each day, the data contains the opening, closing, high, low, and adjusted closing prices, as well as the trading volume. The dataset was downloaded from Yahoo Finance and spans 2011/10/01–2021/10/01. For a fair comparison, the test period for all models is 2020/09/30–2020/12/31, comprising 64 trading days (5,952 daily records). The dataset is publicly available.[1]

Table 1: Technical indicators used as input features.

Indicator	Role
RSI (Relative Strength Index)	strength/direction of price trend
MACD	trend, from two exponential MAs
SAR (Parabolic Stop-and-Reverse)	trend and reversal points
ADX (Average Directional Index)	strength of the trend
Stochastic	momentum vs. support/resistance
MFI (Money Flow Index)	overbought/oversold from price+volume
CCI (Commodity Channel Index)	deviation of price from its mean

5.2 Preprocessing

Feature selection. Beyond the five raw price features and the volume, we follow common practice in technical analysis and augment each stock's daily record with a set of technical indicators that apply mathematical transformations to price and/or volume. We use the indicators listed in Table 1, covering trend, momentum/oscillator, and volume families. We use the signals extracted by these indicators as input features. After computing the features, the label of each day is set according to whether the next day's closing price is higher (class 1) or not (class 0) than the current one.

Plausible label discovery. Because our graph-based models are semi-supervised, predicting the labels of all nodes in a day's graph requires correct labels for a subset of nodes; yet on the target (prediction) day the true next-day labels are unavailable. To address this, we adopt the *plausible label discovery* (PLD) procedure of GCNET [10], which assigns reliable pseudo-labels ("plausible labels") to a subset of nodes. PLD splits the historical data into a training and a validation set, where the validation set consists of the d days immediately before the test period. For each stock, it trains classical, retrospective classifiers (decision tree, naive Bayes, linear and quadratic discriminant analysis, and random forest) and ranks them by their validation performance, selecting the best model per stock to generate that stock's plausible label. Following GCNET, the per-stock predictability score is multiplied by the node's clustering coefficient, under the assumption that nodes with denser neighborhoods are better seeds for propagating information through the network; the highest-scoring nodes are then chosen to receive plausible labels. In our experiments, 20 of the 93 nodes are selected.

[1] https://github.com/ut-kdd/GCNET-Dataset

5.3 Initial Graphs

TGSL can be seeded with any initial graph. We study three choices. *(i) Correlation graph*: a complete graph whose edge weights are the correlation between the price-movement directions of two stocks; following prior work, we also evaluate its maximum spanning tree to prune redundant edges. *(ii) Influence graph*: the influence network introduced by GCNET [10], which on our data contains 93 nodes and 1,355 edges. *(iii) Isolated graph*: a graph with no edges, used to probe how much the initial structure matters.

5.4 Evaluation Metrics

As in prior work, we report two standard classification metrics for the movement-direction task: accuracy and F1-score. We first compute the confusion matrix from each stock's daily prediction and then obtain

$$\text{ACC} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{F1} = \frac{TP}{TP + \frac{1}{2}(FP + FN)} \tag{8}$$

5.5 Baselines

We compare against two families of methods. The first are general-purpose *graph structure learning* models, SUBLIME [16] (unsupervised) and SLAPS [5] (self-supervised), which we adapt to our data using their public implementations with parameters tuned for our setting; for SUBLIME we report the structure-refinement task seeded with the influence graph. Comparing against these clarifies the value of an architecture tailored to temporal data. The second family are *graph-based stock predictors* that use a predefined graph: GCNET [10] (the closest baseline, influence graph + GCN + plausible labels), HATS [13] (relational graph + hierarchical attention), Price Graphs [30] (visibility graph + recurrent/attention layers), CNNpred [8] (CNN over a target series and its three nearest neighbors in the correlation graph), and a GAT [28] that performs node classification over the complete stock graph. Comparing against these clarifies the value of the input graph for graph-based prediction.

5.6 Implementation Details

All models are implemented in PyTorch; GNNs use the DGL library and graph analyses use NetworkX. Every neural component is a two-layer network with 16 hidden units, except the fully connected classifier ($16 \to 2$). The temporal window is $T = 15$ days. All models are trained with Adam (learning rate 0.001, weight decay 0.0005, dropout 0.2). Except for TGSL-S1, which is trained independently per day and therefore uses a fixed number of nodes for validation, we use the last day of the training set as the validation set and keep the checkpoint with the best validation performance. To obtain robust estimates, we run the PLD procedure 5 times to produce 5 independent plausible-label sets, run each model 5 times on each set, and report the mean and standard deviation of accuracy and F1 over the resulting 25 runs.

Table 2: Next-day movement prediction on 93 NASDAQ stocks (mean±std over 25 runs). Best in **bold**, second best underlined.

Model	Accuracy	F1-score
SUBLIME [16]	0.510 ± 0.024	0.437 ± 0.045
SLAPS [5]	0.532 ± 0.016	0.442 ± 0.020
CNNpred [8]	0.534 ± 0.006	0.443 ± 0.015
GAT [28]	0.522 ± 0.003	0.492 ± 0.004
Price Graphs [30]	0.552 ± 0.003	0.461 ± 0.008
HATS [13]	0.551 ± 0.002	0.453 ± 0.007
GCNET [10]	0.567 ± 0.011	$\underline{0.503 \pm 0.003}$
TGSL-S1 (ours)	$\underline{0.577 \pm 0.007}$	0.474 ± 0.027
TGSL-S2 (ours)	0.564 ± 0.008	0.502 ± 0.009
TGSL-P (ours)	0.557 ± 0.017	0.466 ± 0.069
TGSL (ours)	$\mathbf{0.579 \pm 0.006}$	$\mathbf{0.520 \pm 0.012}$

5.7 Results

Table 2 reports the comparison between our models and the baselines. TGSL attains the best accuracy (0.579) and F1 (0.520), improving over the strongest baseline, GCNET (0.567 accuracy, 0.503 F1), by about 1% on both metrics; the improvement is statistically significant. The general-purpose structure-learning baselines SUBLIME and SLAPS perform worst, as they ignore the sequential nature of the data and learn a single static graph; SUBLIME, being fully unsupervised, is the weakest, indicating that label information is essential for structure learning on this task with our features. GAT outperforms some predictors but, despite learning per-edge attention coefficients, cannot create or remove edges and so trails TGSL. CNNpred, HATS, Price Graphs, and GCNET all rely on predefined, static graphs; the superiority of TGSL over them highlights the benefit of *learning* the structure for the input set of stocks.

Effect of the initial graph on TGSL. Table 3 reports TGSL seeded with the isolated, influence, and correlation graphs. The differences are not statistically significant: TGSL reaches essentially the same accuracy regardless of the initial structure, and even the empty (isolated) graph yields the best mean accuracy and the lowest variance. We conclude that TGSL can recover a good structure independently of the input graph, directly addressing the challenge of choosing a stock graph, although a noisy input graph does make convergence somewhat harder.

Table 3: TGSL with different initial graphs.

Initial graph	Accuracy	F1-score
Isolated	0.579 ± 0.006	0.520 ± 0.012
Influence	0.578 ± 0.011	0.514 ± 0.017
Correlation	0.577 ± 0.010	0.502 ± 0.016

Table 4: TGSL-S1 under different training/inference strategies.

Strategy	Accuracy	F1-score
Semi-supervised	0.551 ± 0.010	0.461 ± 0.019
Avg. graph, 3 prev. days (sup.)	0.571 ± 0.005	0.469 ± 0.014
Avg. graph, 5 prev. days (sup.)	0.577 ± 0.007	0.474 ± 0.027
Inferred graph, prev.-day model (sup.)	0.562 ± 0.009	0.462 ± 0.022
Continue prev.-day model (semi-sup.)	0.564 ± 0.007	0.470 ± 0.010

5.8 Ablation: Model Variants

Our four models form an evolution toward the final architecture, which lets us isolate the effect of each component.

TGSL-S1 training/inference strategies. Table 4 compares the strategies for using TGSL-S1 introduced in Section 4.3. The semi-supervised mode is hampered by the noise of the plausible labels. The best results come from supervised training followed by reusing the average of the previous five days' learned graphs for prediction; the five-day window carries more useful information than three days. Continuing to train the previous day's model semi-supervisedly on the target day improves the F1-score by about 1% over inferring the graph directly, by letting the model see the current day's features.

TGSL-S2 and TGSL-P vs. the initial graph. Unlike TGSL, the simpler structure-learning variant TGSL-S2 (Table 5, top) is sensitive to the input graph: the influence graph performs best, then the correlation graph, then the isolated graph: more accurate inputs lead to better predictions. The prediction-only TGSL-P (Table 5, bottom) shows the same ordering and the lowest accuracies overall, as expected for a model that does not learn the structure.

Discussion. Among the proposed models, TGSL performs best, and its advantage is statistically significant. Removing one of its two LSTMs gives TGSL-S2, the second-best model: although its accuracy is 1% below TGSL-S1, its F1-score is about 3% higher, indicating better prediction quality. Further removing the initializer GNN from TGSL-S2 recovers the TGSL-S1 architecture, which operates directly on sequential features; the drop from TGSL-S2 to TGSL-S1 confirms the importance of extracting structural features through the shared initializer GNN applied over the window of graphs. Finally, TGSL-P is the weakest of our models, reaffirming the value of the structure-learning component, yet it still outperforms HATS, Price Graphs, and CNNpred, underlining the benefit of jointly using structural and temporal information even without learning the graph.

Table 5: TGSL-S2 (top) and TGSL-P (bottom) with different initial graphs.

Initial graph	Accuracy	F1-score
TGSL-S2		
Isolated	0.549 ± 0.007	0.489 ± 0.004
Influence	0.564 ± 0.008	0.502 ± 0.009
Correlation	0.550 ± 0.008	0.495 ± 0.007
TGSL-P		
Isolated	0.538 ± 0.016	0.363 ± 0.050
Influence	0.557 ± 0.017	0.466 ± 0.069
Correlation	0.548 ± 0.015	0.390 ± 0.026

5.9 Network Analysis

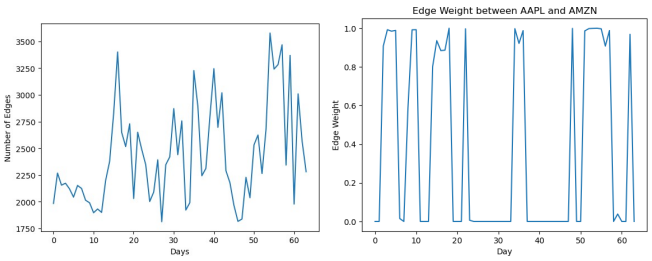
We finally inspect the graphs produced by TGSL, separately for the test and training periods, to verify that the learned structures are meaningful.

Test period. TGSL builds a distinct graph for each of the 64 test days. The number of edges ranges from $1{,}813$ to $3{,}578$ over 93 nodes (Figure 3a), confirming that a different graph is generated per day. Figure 3b shows the time series of the edge weight between Apple (AAPL) and Amazon (AMZN), and Figure 3c the trend of the Frobenius norm of the adjacency matrix, whose average is 16.4. To probe homophily, for every pair of nodes we count how often they share a label over the test period and plot it against their average edge weight (Figure 3d, a random sample of 300 pairs): the upward trend shows that pairs of stocks that more frequently share a movement label are connected with larger weights. The distribution of edge weights (Figure 3e) has mean 0.26 and median 0.16, with extremes at 0 and 1.

Training period. Repeating the analysis on 110 training days, the number of edges ranges from $2{,}140$ to $3{,}065$ and the average Frobenius norm is 30.4; the edge-weight distribution has a lower mean (0.19) and median (0.05) than on the test set. Because the model fits the training data better, the homophily trend is even more pronounced than on the test set. Together, these observations show that TGSL produces sparse, time-varying, and label-homophilic structures rather than collapsing to a trivial or static graph.

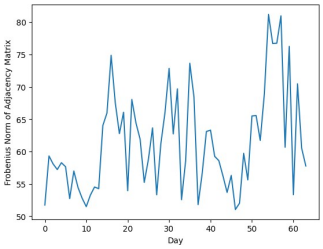
6 Conclusions

We introduced TGSL, a temporal graph structure learning model for stock market prediction whose ultimate goal is to forecast the next-day price-movement direction of each stock. TGSL jointly exploits the temporal information of each stock, through LSTM encoders, and its structural information, through graph neural networks, and is able to produce, for every day, an optimized graph on which the GNN classifier performs well. Two parallel LSTMs, one generating the adjacency matrix and one generating the node embed-

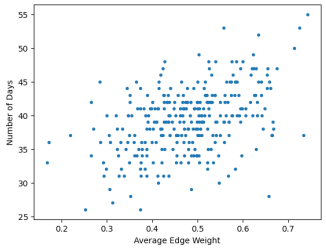


(a) Number of edges per day.

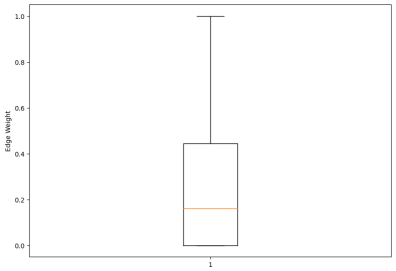
(b) AAPL–AMZN edge weight.



(c) Frobenius norm of $\mathbf{A}$.



(d) Shared labels vs. weight.



(e) Edge-weight distribution.

Figure 3: Analysis of the graphs learned by TGSL on the test period.

dings, give the model more flexibility than its single-LSTM variants, and a label-aware target-smoothing regularizer both improves classification and sparsifies the learned graph. Crucially, TGSL is largely independent of the input graph: it attains its best results even when seeded with an empty graph, thereby resolving the long-standing challenge of choosing a stock graph through learning rather than hand-crafting.

Through an ablation across three progressively simpler variants, we quantified the contribution of each component: removing the second LSTM, the initializer GNN, and finally the structure-learning module each degrades performance, yet even the prediction-only variant outperforms several graph-based predictors that rely on static graphs. On 93 NASDAQ stocks, TGSL improves next-day movement prediction accuracy by 1% over the strongest baseline, and our analysis shows that the learned graphs are sparse, time-varying, and homophilic with respect to movement labels. These results validate our central hypothesis: the graphs chosen in prior network-based predictors are sub-optimal, and learning the structure end-to-end increases both the accuracy and the interpretability of the predictions.

Future work. Recent graph-based stock predictors increasingly incorporate auxiliary signals such as news or social media; extending TGSL to learn *heterogeneous* graphs with multiple node and edge types is a natural and promising direction, as most structure-learning methods (including ours) target homogeneous graphs. Because the prediction loss in our models drives both the classifier and the structure learner, the reliability of the (semi-supervised) labels is critical; obtaining trustworthy labels, choosing which nodes to label, or removing the dependence on labels altogether via self-supervised structure learning all merit further study. Higher-quality node embeddings, e.g., from transformer-based encoders, would likely yield higher-quality graphs, especially for metric-learning constructions. A fundamental difficulty remains the absence of a ground-truth optimal graph, which complicates both learning and evaluation; as a step in this direction we experimented with a "same-label" graph as a proxy ground truth, but more investigation is needed. Finally, the information contained in the learned graphs can itself serve as input to other models, for instance, by extracting features from the time series of the edge weight between two stocks.

References

[1] Yu Chen, Lingfei Wu, and Mohammed Zaki. Iterative deep graph learning for graph neural networks: Better and robust node embeddings. *Advances in Neural Information Processing Systems*, 33:19314–19326, 2020.

[2] Luca Cosmo, Anees Kazi, Seyed-Ahmad Ahmadi, Nassir Navab, and Michael Bronstein. Latent patient network learning for automatic diagnosis. *arXiv preprint arXiv:2003.13620*, 2020.

[3] Hilmi E Egilmez, Eduardo Pavez, and Antonio Ortega. Graph learning from data under Laplacian and structural constraints. *IEEE Journal of Selected Topics in Signal Processing*, 11(6):825–841, 2017.

[4] Frank Emmert-Streib, Shailesh Tripathi, Olli Yli-Harja, and Matthias Dehmer. Understanding the world economy in terms of networks: a survey of data-based network science approaches on economic networks. *Frontiers in Applied Mathematics and Statistics*, 4:37, 2018.

[5] Bahare Fatemi, Layla El Asri, and Seyed Mehran Kazemi. SLAPS: Self-supervision improves structure learning for graph neural networks. *Advances in Neural Information Processing Systems*, 34:22667–22681, 2021.

[6] Paweł Fiedor. Networks in financial markets based on the mutual information rate. *Physical Review E*, 89(5):052801, 2014.

[7] Luca Franceschi, Mathias Niepert, Massimiliano Pontil, and Xiao He. Learning discrete structures for graph neural networks. In *International Conference on Machine Learning*, pages 1972–1982. PMLR, 2019.

[8] Ehsan Hoseinzade and Saman Haratizadeh. CNNpred: CNN-based stock market prediction using several data sources. *Expert Systems with Applications*, 129:273–285, 2019.

[9] Vaishali Ingle and Sachin Deshmukh. Hidden Markov model implementation for prediction of stock prices with tf-idf features. In *Proceedings of the International Conference on Advances in Information Communication Technology & Computing*, pages 1–6, 2016.

[10] Alireza Jafari and Saman Haratizadeh. GCNET: Graph-based prediction of stock price movement using graph convolutional network. *Engineering Applications of Artificial Intelligence*, 116:105452, 2022.

[11] Wei Jin, Yao Ma, Xiaorui Liu, Xianfeng Tang, Suhang Wang, and Jiliang Tang. Graph structure learning for robust graph neural networks. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 66–74, 2020.

[12] Arash Negahdari Kia, Saman Haratizadeh, and Saeed Bagheri Shouraki. A hybrid supervised semi-supervised graph-based model to predict one-day ahead movement of global stock markets and commodity prices. *Expert Systems with Applications*, 105:159–173, 2018.

[13] Raehyun Kim, Chan Ho So, Minbyul Jeong, Sanghoon Lee, Jinkyu Kim, and Jaewoo Kang. HATS: A hierarchical graph attention network for stock movement prediction. *arXiv preprint arXiv:1908.07999*, 2019.

[14] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.

[15] Indu Kumar, Kiran Dogra, Chetna Utreja, and Premlata Yadav. A comparative study of supervised machine learning algorithms for stock market trend prediction. In *2018 Second International Conference on Inventive Communication and Computational Technologies (ICICCT)*, pages 1003–1007. IEEE, 2018.

[16] Yixin Liu, Yu Zheng, Daokun Zhang, Hongxu Chen, Hao Peng, and Shirui Pan. Towards unsupervised deep graph structure learning. In *Proceedings of the ACM Web Conference 2022*, pages 1392–1403, 2022.

[17] Rosario N Mantegna. Hierarchical structure in financial markets. *The European Physical Journal B-Condensed Matter and Complex Systems*, 11(1):193–197, 1999.

[18] Christopher Olah. Understanding LSTM networks. `https://colah.github.io/posts/2015-08-Understanding-LSTMs/`, 2015.

[19] Pratik Patil, Ching-Seh Mike Wu, Katerina Potika, and Marjan Orang. Stock market prediction using ensemble of graph theory, machine learning and deep learning models. In *Proceedings of the 3rd International Conference on Software Engineering and Information Management*, pages 85–92, 2020.

[20] Shuyi Peng. *Stock Forecasting using Neural Network with Graphs*. PhD thesis, University of York, 2020.

[21] Leonidas Sandoval Jr. Structure of a global network of financial companies based on transfer entropy. *Entropy*, 16(8):4443–4482, 2014.

[22] Sumeet Sarode, Harsha G Tolani, Prateek Kak, and C S Lifna. Stock price prediction using machine learning techniques. In *2019 International Conference on Intelligent Sustainable Systems (ICISS)*, pages 177–181. IEEE, 2019.

[23] Thomas Schreiber. Measuring information transfer. *Physical Review Letters*, 85(2):461, 2000.

[24] Sukhman Singh, Tarun Kumar Madan, Jitendra Kumar, and Ashutosh Kumar Singh. Stock market forecasting using machine learning: Today and tomorrow. In *2019 2nd International Conference on Intelligent Computing, Instrumentation and Control Technologies (ICICICT)*, volume 1, pages 738–745. IEEE, 2019.

[25] Yoojeong Song and Jongwoo Lee. Design of stock price prediction model with various configuration of input features. In *Proceedings of the International Conference on Artificial Intelligence, Information Processing and Cloud Computing*, pages 1–5, 2019.

[26] Payal Soni, Yogya Tewari, and Deepa Krishnan. Machine learning approaches in stock price prediction: A systematic review. *Journal of Physics: Conference Series*, 2161:012065, 2022.

[27] Prakhar Vats and Krishna Samdani. Study on machine learning techniques in financial markets. In *2019 IEEE International Conference on System, Computation, Automation and Networking (ICSCAN)*, pages 1–5. IEEE, 2019.

[28] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. Graph attention networks. *arXiv preprint arXiv:1710.10903*, 2017.

[29] Anoushka Vyas and Sambaran Bandyopadhyay. Dynamic structure learning through graph neural network for forecasting soil moisture in precision agriculture. *arXiv preprint arXiv:2012.03506*, 2020.

[30] Junran Wu, Ke Xu, Xueyuan Chen, Shangzhe Li, and Jichang Zhao. Price graphs: Utilizing the structural information of financial time series for stock prediction. *Information Sciences*, 588:405–424, 2022.

[31] Hui Xu, Liyao Xiang, Jiahao Yu, Anqi Cao, and Xinbing Wang. Speedup robust graph structure learning with low-rank information. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*, pages 2241–2250, 2021.

[32] Wentao Xu, Weiqing Liu, Lewen Wang, Yingce Xia, Jiang Bian, Jian Yin, and Tie-Yan Liu. HIST: A graph-based framework for stock trend forecasting via mining concept-oriented shared information. *arXiv preprint arXiv:2110.13716*, 2021.

[33] Liang Yang, Zesheng Kang, Xiaochun Cao, Di Jin, Bo Yang, and Yuanfang Guo. Topology optimization based graph convolutional network. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 4054–4061, 2019.

[34] Tao Yin, Chenzhengyi Liu, Fangyu Ding, Ziming Feng, Bo Yuan, and Ning Zhang. Graph-based stock correlation and prediction for high-frequency trading systems. *Pattern Recognition*, 122:108209, 2022.

[35] Tao You, Paweł Fiedor, and Artur Hołda. Network analysis of the Shanghai stock exchange based on partial mutual information. *Journal of Risk and Financial Management*, 8(2):266–284, 2015.

[36] Xiang Zhang and Marinka Zitnik. GNNGuard: Defending graph neural networks against adversarial attacks. *Advances in Neural Information Processing Systems*, 33:9263–9275, 2020.

[37] Jie Zhou, Ganqu Cui, Shengding Hu, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun. Graph neural networks: A review of methods and applications. *AI Open*, 1:57–81, 2020.

[38] Yanqiao Zhu, Weizhi Xu, Jinghao Zhang, Qiang Liu, Shu Wu, and Liang Wang. Deep graph structure learning for robust representations: A survey. *arXiv preprint arXiv:2103.03036*, 2021.